

Digital Electronics

C language tutorial

Maurizio MARTINA
Electronics and Telecommunications Department
Politecnico di Torino
maurizio.martina@polito.it

Disclaimer

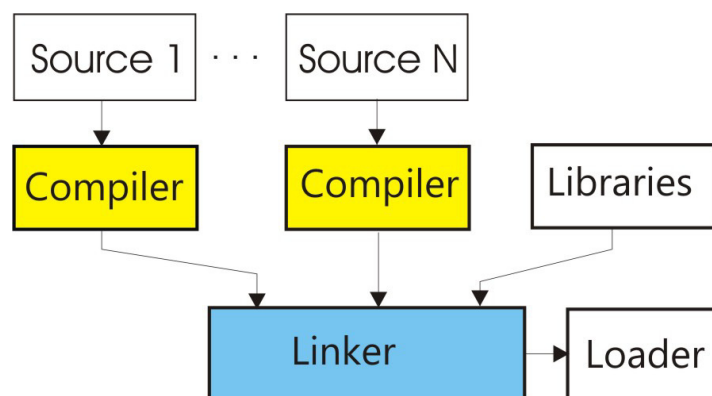
Part of the material in this presentation is taken from the material prepared by Prof. F. Gregoretti and Dr. Mario Nicola.

History

- The C language has been defined in early '70s by Dennis Ritchie at the AT&T labs.
- Objectives:
 - Having a sufficiently high-level language to improve readability.
 - Having an hardware-independent language.
 - Being able to generate efficient binary code.

3

Binary code generation



4

Design Flow

- To develop your application you ought to follow a proper design flow:

- Write C code
- Compile
- Download
- Execute

On the host PC

On the board

5

Variables

2's complement	{	char	character – a single byte
		int	integer
		short	short integer
		long	long integer
Hardware support ?	{	float	floating point
		double	double precision float

char → 8 bits
 $\text{short} \leq \text{int} \leq \text{long}$

6

Variable declaration and use

```
int      celsius, index;
float    fahr;

celsius  = 20; // This is a comment
index    = 0;

/*
This is multi-line comment
*/

fahr = 0.786;
```

7

Constants

Integer constants be expressed in decimal, octal or hexadecimal:

1234	Decimal	Is the default base
12347L	Long Decimal	
0177	octal	A leading 0 determines the base
0x7aaa	Hexadecimal	A leading 0x determines the base

Character constants

'a'	represents the code of ASCII character 0 (48)
'\n'	escape sequence for newline

8

Note

- Since the `char` type is one byte, it can be used to represent either a character or an 8-bit value.
- Examples:

```
char val = '0'; // ASCII value (48)
char num = 0; // number (0)
char num = 127; // 2's compl. max (pos.)
char num = -128; // 2's compl. min (neg.)
unsigned char num = 255; // unsigned max
```

9

Symbolic constants

```
#define L_LIMIT 0 /* defines an integral
                  constant */
#define MAXTEMP 24.6 /* defines a float
                     constant */
```

At compile time

Improve program readability and maintainability

10

Preprocessor - I

- The code shown in the previous example exploits directives, which are evaluated before starting the real compilation process.
- Preprocessor directives:
 - `#include <file>`
 - `#include "file"`
 - First the content of `file` is added to the code and then the compilation starts.

11

Preprocessor - II

- Other preprocessor directives:
 - `#define MYCONST 38`
 - Every occurrence of the symbol `MYCONST` is substituted with `38`.
 - If we change the value of the constant, we need to recompile the code to see the effect.
- Conditional compilation:
 - Through the preprocessor directives we can include/exclude the compilation of portions of code.

12

Preprocessor - III

```
#ifdef MYCONST
/* some code ... */
#endif
```

- The code is compiled only if the symbol MYCONST is defined.

```
#ifndef MYCONST
/* some code ... */
#endif
```

- The code is compiled only if the symbol MYCONST is not defined.

13

Arrays

```
int digit[10] /* Array of integers
              index of the 1st is 0
              index of last is 9 */
```

```
#define MAXLEN 1000;
```

```
char line[MAXLEN];
```

14

Bitwise operators

Applicable only to integral operands.

Unary

`~` one's complement (not)

Binary

`a << b` left shift of a by b bits (multiply by 2^b)

`a >> b` right shift (divide by 2^b)

`a & b` bitwise AND of a and b

`a | b` bitwise OR of a and b

`a ^ b` bitwise XOR of a and b

15

Bit operators examples

```
char    a,b,c;
c = a | 0x4;           // set bit 2 of a
b = b & ~(0x4);        // clear bit 2 of b
c = ~(a<<3);           /* shift-right by 3 bits, then
                        complement and put in c */
.....
```

16

Operators

- Arithmetic Operators
- Relational and Logical Operators
- Increment and Decrement operators
- Bitwise operators
- Assignment Operators

17

Arithmetic Operators

Integer and floating point if hardware unit available

- addition +
- subtraction -
- multiplication *
- division /
- modulus % (remainder – not float or double)

`c = a%b;`

All other functions are *library* functions

18

Cost of arithmetic operators

- Integer arithmetic Fast
 - Floating point arithmetic in hardware Slower
 - Floating point arithmetic in software Very slow
- + , -
*
/
- ↓
- Library functions Very very slow
sqrt, sin, log, etc..

19

Simple Benchmarks (1)

```
for (i=0; i < 10000; ++i)
{ /* arithmetic operation */
```

Pentium 4 with good hardware support

Operator	Time	Operator	Time
+ (int)	1	+ (double)	5
* (int)	5	* (double)	5
/ (int)	12	/ (double)	10
		sqrt	28
		sin	48
		pow	275

20

Simple Benchmarks (2)

400 MHz ARM Processor

Operator	Time	Operator	Time
+ (int)	1	+ (double)	140
* (int)	1	* (double)	110
/ (int)	7	/ (double)	220
<< (int)	2	sqrt	500
		sin	3300
		pow	820

21

C arithmetic lessons

- Try to use integer addition/subtraction;
- Avoid multiplication unless you have hardware;
- Avoid division;
- Avoid floating point, unless you have hardware;
- Really avoid mathematical library functions;

22

Assignment operators

The basic assignment statement `=` may be associated to an operator into an assignment operator:

`expr1 op= expr2` is equivalent to

`expr1 = expr1 op expr2`

`i += 2;` corresponds to

`i = i+2;`

`x *= y+1;` corresponds to

`x = x*(y+1);`

23

Examples

```
char a, b, c, d, e;
```

```
... // variables are initialized
```

```
a |= 0x4;          /* set bit 2 of a      */
```

```
b &= ~0x4;         /* clear bit 2 of b    */
```

```
c &= ~(1 << 3) /* clear bit 3 of c    */
```

```
d ^= (1 << 5) /* toggle bit 5 of d   */
```

```
e >>= 2;          /* divide by 4          */
```

24

Faking multiplication

Addition, subtraction and shifting are fast. Some time can be used to replace multiplication. Not all processors have a hardware multiplier.

Recall the multiplication algorithm from elementary school but think binary

$$\begin{array}{r}
 101011 \times 1101 \\
 \hline
 101011 \\
 10101100 \\
 101011000 \\
 \hline
 1000101111
 \end{array}$$

$43 \times 13 =$
 $43 + 43 \ll 2 + 43 \ll 3 = 559$

Useful for

- multiplication by constants
- “simple” multiplicands
- when no hardware multiplier is available

25

Faking division

Division is a much more complicated algorithm that involves decisions but a division by a power of two is just a shift:

$$a/2 = a \gg 1$$

$$a/4 = a \gg 2$$

$$a/8 = a \gg 3$$

There is no general shift and add replacement for division but sometimes it may be turned into a multiplication:

$$a / 1.333333$$

$$= a * 0.75$$

$$= a * 0.5 + a * 0.25$$

$$= a \gg 1 + a \gg 2$$

26

Relational and Logical Operators

- greater than >
- greater equal than >=
- lower than <
- lower equal than <=
- equal ==
- different !=

(i <= limit-1) **boolean expression !**

27

Increment and Decrement Operators

- increment by 1 ++
- decrement by 1 --

Can be prefixed or post fixed

If we write

```
if (n == 5) {
    x = n++;
    y = n; }    // x equal to 5 and y equal to 6
```

But if we write

```
if (n == 5) {
    x= ++n;
    y=n; }    // both x and y equal to 6
```

They are of particular interest when applied to **pointers ...**

28

Functions

Functions allow to break large and complex computing tasks into smaller ones. They allow for:

- Managing complexity;
- Re-using code.

C has been designed to make function use easy and efficient.

A function has **parameters** (optional) performs an **operation** and returns a **value** (optional).

29

Function declaration

A function declaration is:

```
return_type function_name(argument declaration)  
{  
    declarations and statements  
}
```

Example:

```
double    power(double base, int exponent)  
{  
    ...  
}
```

30

Function use

Since source modules may be compiled separately a function must be declared prior to its use.

```
double      power(double base, int exponent);  
double      source;  
double      result,  
int         exp;  
.  
.  
.  
result = power(source, exp);
```

declaration

use

31

The **main()** function

The main function is the top level function and corresponds to the program invocation.

32

Program structure

```
external declarations
main
{
    declarations
    assignment statements
    function calls
    control statements
}
```

33

Parameter passing (1)

The standard parameter passing in C is **by value**
input parameters are copied from the caller into the function and results are copied into the caller specified variable

The function has no access to the caller variables.
Guarantees that the called function does not modify the original caller parameters but it requires a **copy**

- **Overhead**
- **Not suited for passing large data structures**

34

Variable scope and lifetime

- *Automatic* variables:
 - declared inside a function (or context) and visible only to the function (or context).
 - created at function activation and destroyed when the function ends.
 - *Global* variables:
 - declared outside any function and visible to all functions in the same source file.
 - lifetime spans all the execution of the code.
-

35

Modifying scope and lifetime

A global variable can be made visible to other source files which declare it as *external*.

An variable declared *static* is still private to the context where it has been declared, but it is destroyed at the end of the execution.

A *volatile* variable can change its value between different accesses, even if it does not appear to be modified.

36

Examples: static

```
void myfunc(int val)          void main()
{
    static int cnt = 0;
    if (val == 0)
        cnt++;
    ...
}

x=5;
myfunc(x); // cnt = 0
x=0;
myfunc(x); // cnt = 1
x=1;
myfunc(x); // cnt = 1
x=0;
myfunc(x); // cnt = 2
...
}
```

37

Example: volatile

- A memory location is written by something outside the CPU (peripheral).
- The program only reads the value of the memory location → the optimization during compilation may try to remove the access to a never-written memory location !
- The *volatile* attribute tells the compiler to do not touch the variable during the optimization.

38

Control Statements

- **Conditional statement**

- `if - else`
- `if - else if - else`
- `switch - case`

- **Loop statements**

- `while`
- `for`
- `do ... while`

39

if ... else (binary decision)

```

if (expr)           Expression is true if does not have a 0 value
    statement1
else               else may be missing – Careful!
    statement 2
  
```

The *expression* `expr` is evaluated;
 if it is **TRUE** (that is, if `expr` has a nonzero value), `statement1` is executed.
 If it is **FALSE** (`expr` is zero) and if there is an else part, `statement2` is executed instead.

40

If ... else

BE CAREFUL about the possible omission of `else` because it may lead to errors

```
if (n > 0)
    if (s[i] > 0)
    {
        printf (...);
        return i;
    }
else
    printf ("error \n");
```

Indentation does not correspond to semantics !

41

How to obtain what we want ...

```
if (n > 0)
{
    if (s[i] > 0)
    {
        printf (...);
        return i;
    }
}
else
    printf ("error \n");
```

42

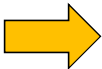
Conditional expression

It has the following form:

expr1 ? expr2 : expr3

and evaluates as follows:

- *expr1* is evaluated;
- If it is **TRUE** then the result is *expr2* otherwise it is *expr3*

z = (a > b) ? a : b; 

```
if (a > b)
    z=a;
else
    z=b;
```

43

Multi way decision: if ... else if ..

```
if (expression)
    statement
else if (expression)
    statement
else if (expression)
    statement
else if (expression)
    statement
else
    statement
```

44

Multi way decision: switch ..

Matches an expression with a number of *constant* integer values

```
switch (expression) {  
    case const-expr: statements  
    case const-expr: statements  
    case const-expr: statements  
    default: statements  
}
```

45

Multi-way branches

```
if (a==1)  
    foo();  
else if (a == 2)  
    bar();  
else if (a == 3)  
    baz();  
else if (a == 4)  
    qux();  
else if (a == 5)  
    quux();  
else if (a == 6)  
    corge();
```

```
switch(a) {  
    case 1:  
        foo(); break;  
    case 2:  
        bar(); break;  
    case 3:  
        baz(); break;  
    case 4:  
        qux(); break;  
    case 5:  
        quux(); break;  
    case 6:  
        corge(); break; }
```

46

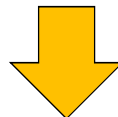
while

```
while (expression)
    statement
```

47

for ..

```
for (expr1; expr2; expr3)
    statement
```



```
expr1;
while (expr2) {
    statement
    expr3;
}
```

48

do .. while

```
do
    statement
while (expression);
```

The statement is evaluated until the expression becomes false → it is evaluated at least once !

49

Pointers

- Variables are locations in memory.
- Processors access to variables in memory by providing their **addresses**.
- **Pointers** are variables (or constants) which contain the **address** of another variable.

50

Pointer Declaration

- The dereferencing operator `*` is used both to declare a pointer and to access to the content of the pointer.
- The reference operator `&` is used to the address of variable and to store it in a pointer.

Example:

```
char b; // b is a variable of type char
char *p; // p is a pointer of type char

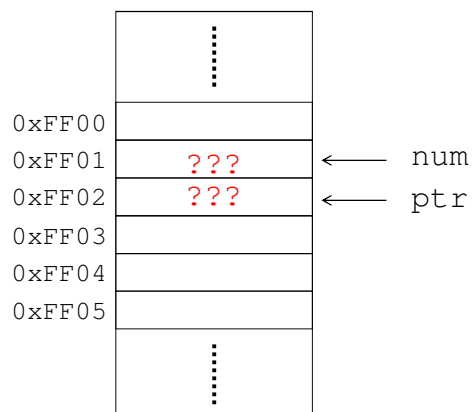
b = *p; // b takes the value of the variable pointed by p
p = &b; // p takes the address of variable b
```

51

Example - I

```
int num;
int *ptr;
```

```
ptr = &num;
num = 37;
(*ptr)++;
ptr++;
```

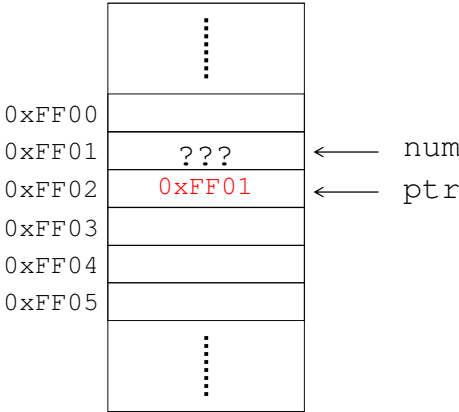


52

Example - II

```
int num;  
int *ptr;
```

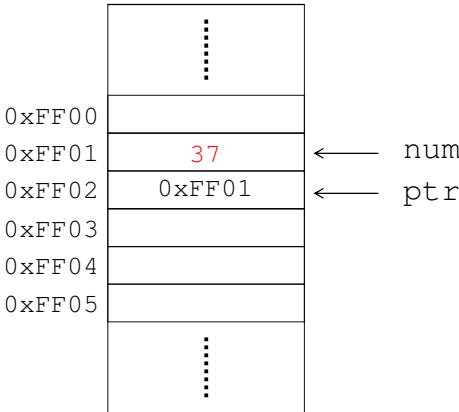
```
ptr = &num;  
num = 37;  
(*ptr)++;  
ptr++;
```



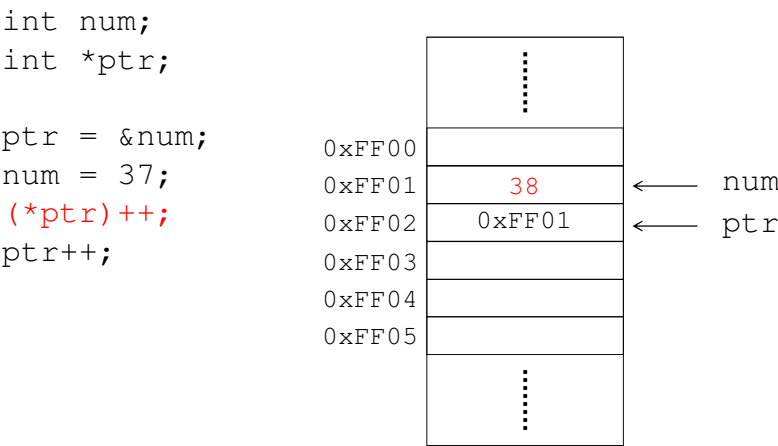
Example - III

```
int num;  
int *ptr;
```

```
ptr = &num;  
num = 37;  
(*ptr)++;  
ptr++;
```

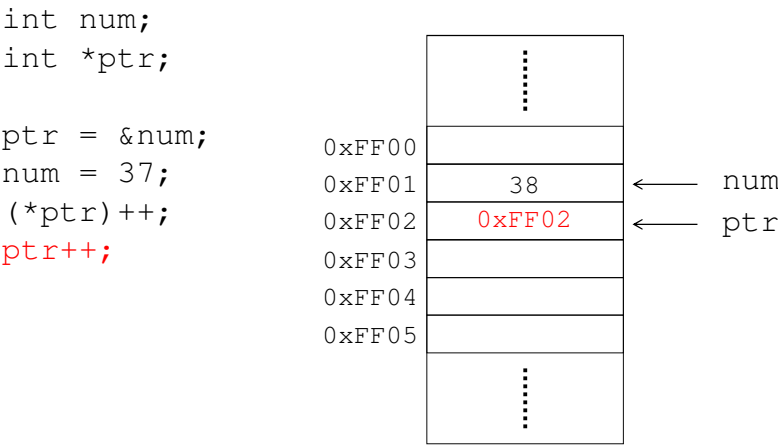


Example - IV



55

Example - V



56

Pointer and arrays

An array name is equivalent to a pointer to its first element

```
int list[10];  
int *ptr;  
  
ptr = &(list[0]);  
ptr = list;
```

57

Passing parameters by reference

- Passing a pointer as a parameter of a function allows to pass parameters by reference:
 - the called function may have access to caller internal variables.

58

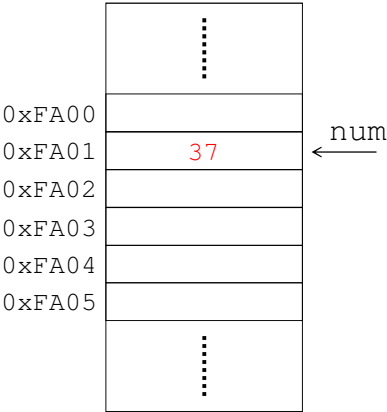
Example - I

```
void incr_no_work(int a)
{
    a++;
}

void incr_work(int *a)
{
    (*a)++;
}

int main()
{
    int num = 37;

    incr_no_work(num);
    incr_work(&num);
}
```



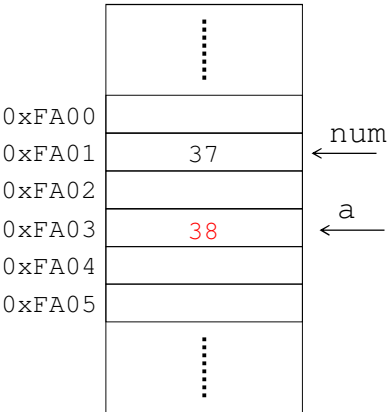
Example - II

```
void incr_no_work(int a)
{
    a++;
}

void incr_work(int *a)
{
    (*a)++;
}

int main()
{
    int num = 37;

    incr_no_work(num);
    incr_work(&num);
}
```



Example - III

```

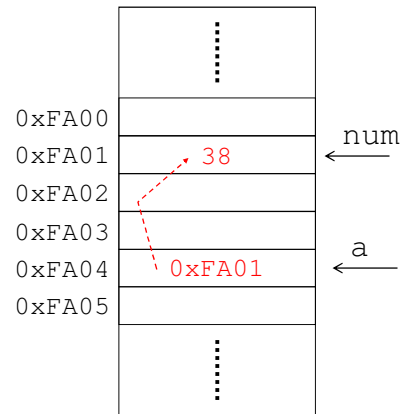
void incr_no_work(int a)
{
    a++;
}

void incr_work(int *a)
{
    (*a)++;
}

int main()
{
    int num = 37;

    incr_no_work(num);
    incr_work(&num);
}

```



61

Data allocation

- Data allocation can be:
 - *static*, the number of required elements is known at compile time.
 - *dynamic*, the number of required elements is known at run-time.
- Static allocation:
 - `int data[10];`
- Dynamic allocation:
 - use of library functions (e.g. `malloc`, `free`).

62

The *malloc* function

- It is defined in *stdlib.h*.
- Prototype:
 - `void * malloc(size_t size);`
- `malloc` returns:
 - a pointer to a chunk of contiguous memory to store `size` bytes;
 - `NULL` if there is not enough memory.
- The return pointer:
 - is `void` (unspecified type);
 - casting is strongly advised:
 - `int *ptr`
 - `ptr = (int *)malloc(...);`

63

Example

- Use a pointer instead of a static array:
 - `int *vect;`
- Allocate the required number of elements using the `malloc` function:
 - `vect = (int *)malloc(nelems * sizeof(int));`
- The use is the same as for static arrays.
- When it is no more used, free the memory:
 - `free(vect);`

64

Comparing Storage classes

- On most processors, access to automatic (stacked) data and globals is equally fast.
- Automatic usually preferable since the memory is reused when function terminates.
- Danger of exhausting stack space with recursive algorithms (recursion is not used in most embedded systems).
- The heap (malloc) should be avoided if possible:
 - Allocation/deallocation is unpredictably slow;
 - Danger of exhausting memory;
 - Danger of fragmentation;
- Best used sparingly in embedded systems.

65

User-defined types

- In C the keyword **typedef** is used to build user-defined types.
- It can be used to:
 - create an alias of an already-existing type;
 - build enumerative types;
 - to aggregate already-existing types.

66

typedef - I

- An alias of an existing type can be used as type as well:
 - `typedef int StudentID;`
 - The new type `StudentID` behaves as `int`.
- Enumerative types refer to some enumerative constants:
 - `typedef enum {HEARTS, CLUBS, SPADES, DIAMONDS} seed_t;`
 - This statement declares a new type named `seed_t`;
 - A variable of type `seed_t` can take only `HEARTS`, `CLUBS`, `SPADES`, `DIAMONDS`.

Structures

- In C you can build structured resorting to the keyword **struct**. Using *struct* and *typedef* one can build new types of variables.

```
typedef enum {HEARTS, CLUBS, SPADES, DIAMONDS} seed_t;

typedef enum {ONE, TWO, THREE, FOUR, FIVE, SIX, SEVEN,
EIGHT, NINE, TEN, JACK, QUEEN, KING} value_t;

typedef struct
{
    seed_t CardSeed;
    value_t CardValue;
} card_t;
```

struct e typedef

- Once a new type is defined you can declare variables and pointers.
- To access the internal variables of a structure there are two possibilities:
 - From variable: var_name.internal_variable;
 - From pointer: ptr_name->internal_variable.

Example

```
card_t var;  
card_t *ptr;  
ptr=&var;  
  
var.CardSeed = HEARTS;  
var.CardValue = JACK;  
  
ptr->CardSeed = CLUBS;  
ptr->CardValue = KING;
```

70